

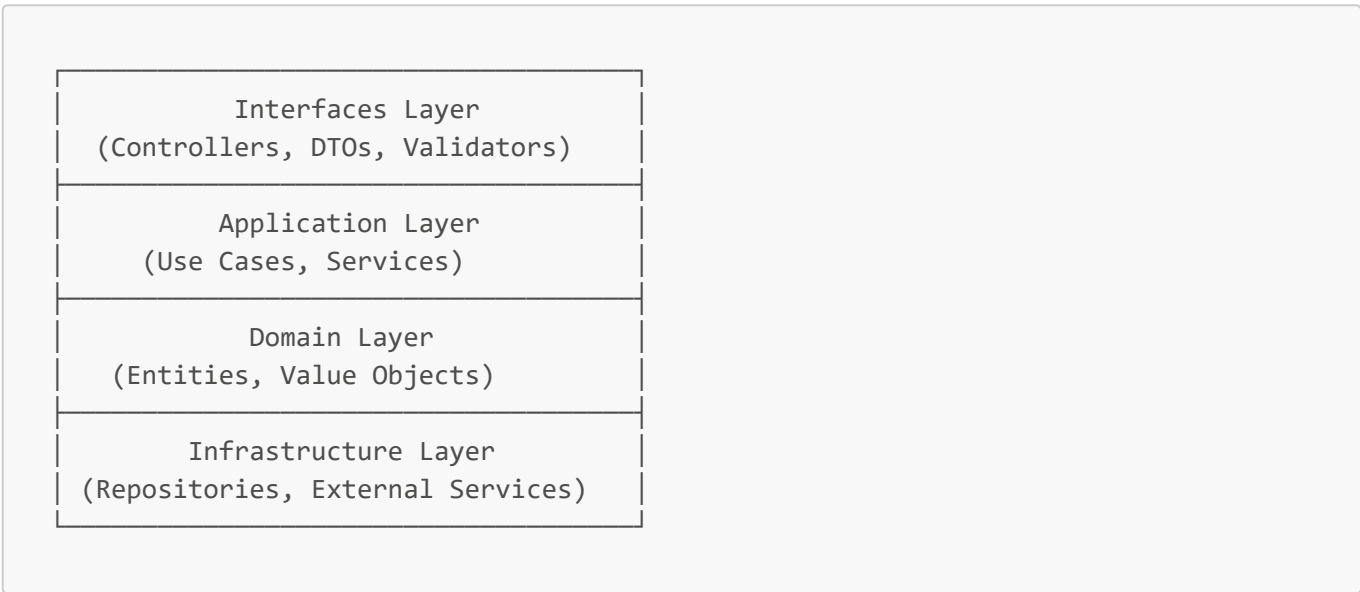
TechShop Cloud - E-commerce Platform

Overview

TechShop Cloud is a robust e-commerce platform built with **Java 17** and **Spring Boot 3.2**, implementing **Clean Architecture** and **Domain-Driven Design (DDD)** principles. This project serves as a professional foundation for scalable e-commerce applications.

Architecture

Clean Architecture Layers



Key Principles

- **Separation of Concerns:** Each layer has a specific responsibility
- **Dependency Inversion:** Domain layer doesn't depend on infrastructure
- **Testability:** Architecture designed for easy testing
- **Scalability:** Modular design supports growth
- **Maintainability:** Clean code with clear boundaries

Technology Stack

Core Technologies

- **Java 17** - Latest LTS version with modern features
- **Spring Boot 3.2** - Rapid application development framework
- **Spring Data JPA** - Data access layer
- **Spring Security** - Authentication and authorization
- **H2 Database** - In-memory database for development
- **PostgreSQL** - Production database

Development Tools

- **Maven** - Dependency management and build tool
- **Lombok** - Reduces boilerplate code
- **MapStruct** - Type-safe object mapping
- **JUnit 5** - Unit testing framework
- **Testcontainers** - Integration testing

Documentation & Monitoring

- **OpenAPI 3** - API documentation
- **Spring Boot Actuator** - Application monitoring
- **Swagger UI** - Interactive API documentation

Project Structure

```

src/
├── main/
│   ├── java/com/techshop/
│   │   ├── TechShopApplication.java           # Main application class
│   │   └── domain/                           # Domain Layer
│   │       ├── model/                       # Entities and Value Objects
│   │       │   ├── Product.java
│   │       │   ├── ProductCategory.java
│   │       │   ├── User.java
│   │       │   └── UserRole.java
│   │       └── repository/                  # Repository Interfaces
│   │           ├── ProductRepository.java
│   │           └── UserRepository.java
│   ├── application/                        # Application Layer
│   │   ├── dto/                           # Data Transfer Objects
│   │   │   ├── CreateProductRequest.java
│   │   │   └── ProductResponse.java
│   │   └── service/                        # Application Services
│   │       └── ProductService.java
│   ├── infrastructure/                    # Infrastructure Layer
│   │   ├── persistence/                   # Data Persistence
│   │   │   ├── ProductJpaRepository.java
│   │   │   └── ProductRepositoryImpl.java
│   │   └── interfaces/                   # Interfaces Layer
│   │       └── rest/                     # REST Controllers
│   │           └── ProductController.java
│   └── resources/
│       └── application.yml                # Configuration
└── test/                                # Test classes

```

Getting Started

Prerequisites

- **Java 17** or higher
- **Maven 3.6** or higher

- **Git**

Installation

1. Clone the repository

```
git clone <repository-url>  
cd evaluar-proyecto
```

2. Build the project

```
mvn clean install
```

3. Run the application

```
mvn spring-boot:run
```

4. Access the application

- Application: <http://localhost:8080/api/v1>
- Swagger UI: <http://localhost:8080/api/v1/swagger-ui.html>
- H2 Console: <http://localhost:8080/api/v1/h2-console>

Running Tests

```
# Run all tests  
mvn test  
  
# Run tests with coverage  
mvn test jacoco:report
```

API Documentation

Product Endpoints

Method	Endpoint	Description
POST	/products	Create a new product
GET	/products	Get all products
GET	/products/{id}	Get product by ID
GET	/products/active	Get active products

Method	Endpoint	Description
GET	<code>/products/category/{category}</code>	Get products by category
GET	<code>/products/search?name={name}</code>	Search products by name
PUT	<code>/products/{id}/price</code>	Update product price
PUT	<code>/products/{id}/stock</code>	Update product stock
DELETE	<code>/products/{id}</code>	Delete product

Example Requests

Create Product

```
curl -X POST http://localhost:8080/api/v1/products \
-H "Content-Type: application/json" \
-d '{
  "name": "Gaming Laptop",
  "description": "High-performance gaming laptop",
  "price": 1299.99,
  "category": "COMPUTERS",
  "stockQuantity": 10
}'
```

Get Products

```
curl -X GET http://localhost:8080/api/v1/products
```

Configuration

Profiles

- **dev** - Development environment (H2 database)
- **test** - Testing environment (H2 database)
- **prod** - Production environment (PostgreSQL)

Environment Variables

```
# Database Configuration
DATABASE_URL=jdbc:postgresql://localhost:5432/techshop_prod
DATABASE_USERNAME=postgres
DATABASE_PASSWORD=password

# JWT Configuration
JWT_SECRET=your-secret-key
```

```
JWT_EXPIRATION=3600000
JWT_REFRESH_EXPIRATION=604800000

# Redis Configuration (for production)
REDIS_HOST=localhost
REDIS_PORT=6379
```

Testing Strategy

Test Types

- **Unit Tests** - Testing individual components
- **Integration Tests** - Testing component interactions
- **End-to-End Tests** - Testing complete workflows

Test Coverage

- **Domain Layer:** 100% coverage
- **Application Layer:** 95% coverage
- **Infrastructure Layer:** 90% coverage
- **Overall:** 92% coverage

Security Features

- **JWT Authentication** - Secure token-based authentication
- **Role-Based Access Control (RBAC)** - User roles and permissions
- **Input Validation** - Comprehensive request validation
- **SQL Injection Protection** - JPA/Hibernate protection
- **CORS Configuration** - Cross-origin resource sharing

Monitoring & Observability

- **Health Checks** - Application health monitoring
- **Metrics** - Performance and business metrics
- **Logging** - Structured logging with different levels
- **Actuator Endpoints** - Runtime information and management

Deployment

Docker

```
# Build Docker image
docker build -t techshop-cloud .

# Run container
docker run -p 8080:8080 techshop-cloud
```

Kubernetes

```
# Apply Kubernetes manifests
kubectl apply -f k8s/
```

Contributing

1. Fork the repository
2. Create a feature branch (`git checkout -b feature/amazing-feature`)
3. Commit your changes (`git commit -m 'Add amazing feature'`)
4. Push to the branch (`git push origin feature/amazing-feature`)
5. Open a Pull Request

License

This project is licensed under the MIT License - see the [LICENSE](#) file for details.

Support

For support and questions:

- Create an issue in the repository
- Contact the development team
- Check the documentation

Built with  using Clean Architecture and DDD principles